

Internship Defense — RAG Contribution

Summary

- Project: Multi-modal college information assistant (voice, character, chat)
- My role: Implemented the Retrieval-Augmented Generation (RAG) portion only.
- Not my responsibilities: TTS (text-to-speech), STT (speech-to-text), and character/animation components — these were handled by other team members.

Objective

Explain and demonstrate the RAG system I built: how content is indexed, how retrieval works, how it augments generation, and examples showing improvements in factuality and coverage.

My Contribution (Scope)

- Designed and implemented the RAG pipeline used by the assistant.
- Built/documented the indexing process (text extraction, chunking, embeddings, FAISS index creation).
- Implemented the retriever and integration with the generation prompts.
- Tuned retrieval and prompt templates to reduce hallucinations and increase answer relevance.
- Performed basic evaluation (qualitative examples and retrieval checks).

Explicit note: I did NOT implement or modify the project's TTS, STT, or character modules; my contributions are limited to the RAG/retrieval and prompt side.

System Overview (RAG)

Implementation Details

- Chunking: chunks sized to fit model context limits while preserving sentence boundaries.
- Embeddings: used the project's chosen embedding model; embeddings computed and stored alongside index files for speed.
- Indexing: FAISS flat/indexIVF (implementation detail in `vector_store.py`) per topic for faster, focused retrieval.
- Retriever: returns top-N passages with retrieval scores and passage metadata (source, position).
- Prompting: templates prepend a short "context" block with retrieved passages, then a clear instruction to answer using only that context.

Example Query Flow (concise)

Evaluation & Results (high level)

- Qualitative checks: sample queries where answers match source documents and cite passages.
- Retrieval sanity: top results contain verifiable lines from the source documents.
- Metrics: where applicable, measured retrieval recall on held-out queries (basic checks rather than a full benchmark).

Challenges & Mitigations

- Overlap between documents: used metadata and topic-specific indexes to scope retrieval and reduce irrelevant context.
- Long documents: implemented chunking with overlap to preserve context across chunk boundaries.
- Prompt sensitivity: tuned templates and max context tokens to balance helpfulness and hallucination risk.

Files & Locations (reference)

- Indexes: `indexes`
- Retriever: `utils/retriever.py`
- Vector store: `utils/vector_store.py`
- Answer generation: `utils/answer_generator.py`
- Chunking/extraction: `utils/chunker.py`, `utils/pdf_extractor.py`

Conclusion & Future Work

- Conclusion: The RAG pipeline I implemented provides grounded, retrieval-backed answers that improve factual accuracy of the assistant.
- Future work (suggestions):
- Add automated retrieval evaluation (precision/recall) on a labeled dev set.

- Implement reranking or cross-encoder re-ranking for higher precision.
- Add explicit citation formatting and source linking in the UI.

If you want, I can: - Convert this into a PDF or slide deck for your defense. - Add specific example queries and before/after answers showing the benefit of RAG. - Include short code snippets from the key files above.